深度学习基础横纵分析报告

> 从感知机到 Transformer——神经网络如何在七十年间从"实验室玩具"变成驱动全球 AI 的核心引擎。

一、一句话定义

深度学习是用多层神经网络从数据中自动学习特征表示的机器学习方法。它的核心洞察是：**让数据自己说话**——传统机器学习需要人类手工设计特征（feature engineering），深度学习让网络自己从原始数据中逐层抽象出越来越高级的表示。

> 🎯 **读完这篇你能**：理解从感知机到 Transformer 的技术演进逻辑，能解释反向传播为什么是深度学习的"发动机"，知道 CNN/RNN/LSTM/Transformer 分别解决了什么问题。

二、技术背景

2.1 为什么需要"深度"学习?

一个图像分类的例子：区分猫和狗。

- **传统方法**：人工设计特征——边缘检测 → 纹理描述 → 颜色直方图 → 送入 SVM/决策树分类。每一步需要领域专家，换个任务就得重新设计。
- **深度学习**：原始像素 → 第 1 层学边缘 → 第 3 层学纹理 → 第 5 层学耳朵形状 → 第 10 层学"猫"的完整概念。所有层一起学，不需要人工设计。

深度 = 层次化的特征学习。"深"的意义不在于层数多，而在于每层自动学到了更抽象的表示。

三、纵向分析：从感知机到 Transformer

3.1 感知机（1958）—— 一个神经元的梦想

Frank Rosenblatt 在 1958 年建造了 Mark I 感知机——一个能学习区分三角形和正方形的物理机器。从 400 个光传感器输入，经过一层可调权重的"神经元"，输出分类结果。

局限：Minsky & Papert 在 1969 年证明单层感知机连 XOR 都学不会——这个"判决"让神经网络进入了第一个冬天。

3.2 反向传播（1986）—— 深度学习终于有了"发动机"

Rumelhart、Hinton 和 Williams 在 1986 年发表了反向传播算法（backpropagation）。核心思想极其优雅：

1. **前向传播**：输入经过网络，算出预测
2. **计算误差**：预测 vs 真实标签的差值
3. **反向传播**：误差从输出层逐层往回传，用链式求导算出每个参数对误差的贡献
4. **更新参数**：梯度下降——沿着"误差减小最快"的方向微调每个参数

反向传播的本质是"责任的追溯"——一个错误发生时，网络中的每个参数承担多大责任？

3.3 CNN（卷积神经网络）—— 让机器"看见"（2012）

CNN 的核心是三个操作：

- **卷积（Convolution）**：一个小窗口（如 3×3）扫过整张图，检测局部特征（边缘、纹理）。同一个小窗口的参数在所有位置共享——这是 CNN 参数效率的核心
- **池化（Pooling）**：把特征图缩小（如 2×2 取最大值），降低计算量，增加平移不变性
- **全连接层**：最后几层把学到的特征组合起来做分类

AlexNet（2012） 在 ImageNet 上的表现比第二名好了一倍，标志着深度学习时代的真正开端。

3.4 RNN / LSTM —— 让机器理解"序列"（1997–2014）

CNN 处理的是固定大小的输入（一张 224×224 的图片），但自然语言的句子长短不一。

RNN（循环神经网络） 的核心：每个时间步的输出不仅取决于当前输入，还取决于上一步的"隐藏状态"——一个随时间传递的"记忆"。

LSTM（长短期记忆，Hochreiter & Schmidhuber, 1997） 解决了 RNN 的根本问题——长序列带来的梯度消失。引入了三个门：

- **遗忘门**：决定记住什么、忘掉什么
- **输入门**：决定当前输入中哪些值得存储
- **输出门**：决定当前状态中哪些要输出

LSTM 统治了 NLP 近 20 年——机器翻译、语音识别、语言模型都依赖它。但它的致命弱点是**无法并行化**——逐时间步串行计算，速度瓶颈始终无法突破。

3.5 Transformer（2017）—— "Attention Is All You Need"

Transformer 解决了 LSTM 的根本矛盾——如何在保留"关注上下文"能力的同时，实现完全并行的计算。

核心机制是**自注意力（Self-Attention）**：

1. 每个 token 生成三个向量：Query（我想查什么）、Key（我有什么标签）、Value（我的内容）
2. Query 与所有 Key 做点积 → 相似度分数 → softmax → 注意力权重
3. 加权求和 Value → 每个 token 的出向量融合了全序列的信息

为什么是革命：RNN 需要逐时间步串行，"我"要知道"你"的内容得等前面的步骤算完。Transformer 里所有 token 同时互相关注——完全并行，训练速度从"数周"降到"数天"。

详见 Transformer 横纵分析报告 获取完整架构分析。

3.6 Transformer 家族的三大变体（2018）—— 同一个架构，三种用法

Transformer 的原版论文是一个**编码器-解码器（Encoder-Decoder）**架构——编码器处理输入，解码器生成输出。但 2018 年，两个团队各自取了一半，分别开创了整个 NLP 的未来：

```
              Transformer (original, 2017)
              Encoder + Decoder (完整版)
                     /        \
                    /          \
         BERT (2018.10)     GPT (2018.06)
         只用 Encoder        只用 Decoder
         双向理解            单向生成
```

	Transformer (原版)	BERT	GPT
架构	Encoder + Decoder	Encoder-only	Decoder-only
核心任务	序列到序列（翻译）	完形填空（MLM）	下一个词预测（自回归）
上下文方向	编码器双向 + 解码器单向	**双向**（同时看左右）	**单向**（只看左边）
最强能力	翻译、摘要	理解（分类、NER、QA）	**生成（代码、文章、对话）**
生成能力	✅（Decoder 负责）	❌（只能填一个空）	✅（逐 token 生成）
代表模型	T5, BART	BERT, RoBERTa, ModernBERT	GPT-1→GPT-5.5, Llama, Qwen
2026 年状态	特定场景（翻译）	Embedding/分类场景	**主导 LLM 世界**

为什么 GPT（decoder-only）最终统治了世界?

1. **生成是元能力**：你能生成，理解就隐含其中。GPT-3 证明了足够大的 decoder-only 在理解和生成上都能做到 SOTA。但 BERT 天生不会生成。
2. **架构更简洁**：Decoder-only 推理时只有一层自注意力（causal attention），比 encoder-decoder 少了一整个编码器。少一层的代价是速度和显存节省 30-50%。
3. **扩展性更好**：decoder-only 的因果注意力可以复用 KV Cache（前面的 token 不会变，只需要算最后一个的注意力），这对长文本推理至关重要。

但 BERT 没有死——ModernBERT（2024）用 RoPE 和 FlashAttention 把 encoder-only 架构现代化，BGE/E5 等嵌入模型继承了 BERT 的双向编码基因。在"把文本变成向量"这个任务上，encoder-only 仍然是最优选择。

四、横向分析：三大核心架构对比

架构	擅长	核心机制	并行度	衰落原因
CNN	图像/空间数据	局部卷积+共享权重	高	Vision Transformer 挑战
RNN/LSTM	序列（文本/语音）	循环隐藏状态	低	Transformer 替代
Transformer	序列+图像+多模态	自注意力	极高	仍在迭代（Titans 等）

五、关键概念速览

概念	一句话
梯度下降	沿着"误差减小最快"方向更新参数
反向传播	链式法则把误差分配回每个参数
激活函数	给网络加非线性（ReLU/GELU），否则多层=一层
Batch Normalization	每层输入标准化，训练更快更稳定
过拟合	模型背住了训练数据，泛化能力差
Dropout	训练时随机"关掉"部分神经元，防过拟合
学习率	每次更新的步长——太大不收敛，太小太慢
Embedding	把离散 token 映射到连续向量空间

六、延伸阅读

- Transformer 横纵分析报告 — Transformer 架构的完整纵横向分析
- NLP 基础 — 从词向量到预训练模型的 NLP 演进
- 机器学习概览 — 监督/无监督/强化学习的全景框架

本文是横纵分析系列的第 31 篇报告。定位为基础知识，为后续的技术专题（Transformer、MoE、长上下文等）提供入门铺垫。